



Table of Contents

Project Zero: A Template for Other Projects

1.1 On quality

1.1.1 More Reading on Quality

1.2 Suggested project sprints

1.2.1 Inception

1.2.2 Elaboration, part 1: define done

1.2.3 Elaboration, part 2: define components and tests

1.2.4 Construction

1.2.5 Transition

1.3 List of deliverables

1.4 Development tool installation

1.5 Project 0 – Hello World with test cases

1.5.1 Description

1.5.2 Approach

1.5.3 Deliverables

1.5.4 Definition of done

1.6 Summary

1.7 Extras

1.7.1 Static analysis - mypy, flake8

1.7.2 CLI features

1.7.3 Logging

1.7.4 Cookiecutter

Overview of the Projects

2.1 General data acquisition

2.2 Acquisition via Extract

2.3 Inspection

2.4 Clean, validate, standardize, and persist

2.5 Summarize and analyze

2.6 Statistical modeling

2.7 Data contracts

2.8 Summary

Project 1.1: Data Acquisition Base Application

3.1 Description

3.1.1 User experience

3.1.2 About the source data

3.1.3 About the output data

3.2 Architectural approach

3.2.1 Class design

3.2.2 Design principles

3.2.3 Functional design

3.3 Deliverables

3.3.1 Acceptance tests

3.3.2 Additional acceptance scenarios

3.3.3 Unit tests

3.4 Summary

3.5 Extras

3.5.1 Logging enhancements

3.5.2 Configuration extensions

3.5.3 Data subsets

3.5.4 Another example data source

Data Acquisition Features: Web APIs and Scraping

4.1 Project 1.2: Acquire data from a web service

4.1.1 Description

4.1.2 Approach

4.1.3 Deliverables

4.2 Project 1.3: Scrape data from a web page

4.2.1 Description

4.2.2 About the source data

4.2.3 Approach

4.2.4 Deliverables

4.3 Summary

4.4 Extras

4.4.1 Locate more JSON-format data

4.4.2 Other data sets to extract

4.4.3 Handling schema variations

4.4.4 CLI enhancements

4.4.5 Logging

Data Acquisition Features: SQL Database

5.1 Project 1.4: A local SQL database

5.1.1 Description

5.1.2 Approach

5.1.3 Deliverables

5.2 Project 1.5: Acquire data from a SQL extract

5.2.1 Description

5.2.2 The Object-Relational Mapping (ORM) problem

5.2.3 About the source data

5.2.4 Approach

5.2.5 Deliverables

5.3 Summary

5.4 Extras

5.4.1 Consider using another database

5.4.2 Consider using a NoSQL database

5.4.3 Consider using SQLAlchemy to define an ORM layer

Project 2.1: Data Inspection Notebook

6.1 Description

6.1.1 About the source data

6.2 Approach

6.2.1 Notebook test cases for the functions

6.2.2 Common code in a separate module

6.3 Deliverables

6.3.1 Notebook .ipynb file

6.3.2 Executing a notebook's test suite

6.4 Summary

6.5 Extras

6.5.1 Use pandas to examine data

Data Inspection Features

7.1 Project 2.2: Validating cardinal domains — measures, counts, and durations

7.1.1 Description

7.1.2 Approach

7.1.3 Deliverables

7.2 Project 2.3: Validating text and codes — nominal data and ordinal numbers

7.2.1 Description

7.2.2 Approach

7.2.3 Deliverables

7.3 Project 2.4: Finding reference domains

7.3.1 Description

7.3.2 Approach

7.3.3 Deliverables

7.4 Summary

7.5 Extras

7.5.1 Markdown cells with dates and data source information

7.5.2 Presentation materials

7.5.3 JupyterBook or Quarto for even more sophisticated output

Project 2.5: Schema and Metadata

8.1 Description

8.2 Approach

8.2.1 Define Pydantic classes and emit the JSON Schema

8.2.2 Define expected data domains in JSON Schema notation

8.2.3 Use JSON Schema to validate intermediate files

8.3 Deliverables

8.3.1 Schema acceptance tests

8.3.2 Extended acceptance testing

8.4 Summary

8.5 Extras

8.5.1 Revise all previous chapter models to use Pydantic

8.5.2 Use the ORM layer

Project 3.1: Data Cleaning Base Application

9.1 Description

9.1.1 User experience

9.1.2 Source data

9.1.3 Result data

9.1.4 Conversions and processing

9.1.5 Error reports

9.2 Approach

9.2.1 Model module refactoring

9.2.2 Pydantic V2 validation

9.2.3 Validation function design

9.2.4 Incremental design

9.2.5 CLI application

9.3 Deliverables

9.3.1 Acceptance tests

9.3.2 Unit tests for the model features

9.3.3 Application to clean data and create an NDJSON interim file

9.4 Summary

9.5 Extras

9.5.1 Create an output file with rejected samples

Data Cleaning Features

10.1 **Project 3.2: Validate and convert source fields**

10.1.1 **Description**

10.1.2 **Approach**

10.1.3 **Deliverables**

10.2 **Project 3.3: Validate text fields (and numeric coded fields)**

10.2.1 **Description**

10.2.2 **Approach**

10.2.3 **Deliverables**

10.3 **Project 3.4: Validate references among separate data sources**

10.3.1 **Description**

10.3.2 **Approach**

10.3.3 **Deliverables**

10.4 **Project 3.5: Standardize data to common codes and ranges**

10.4.1 **Description**

10.4.2 **Approach**

10.4.3 **Deliverables**

10.5 **Project 3.6: Integration to create an acquisition pipeline**

10.5.1 **Description**

10.5.2 **Approach**

10.5.3 Deliverables

10.6 Summary

10.7 Extras

10.7.1 Hypothesis testing

10.7.2 Rejecting bad data via filtering (instead of logging)

10.7.3 Disjoint subentities

10.7.4 Create a fan-out cleaning pipeline

Project 3.7: Interim Data Persistence

11.1 Description

11.2 Overall approach

11.2.1 Designing idempotent operations

11.3 Deliverables

11.3.1 Unit test

11.3.2 Acceptance test

11.3.3 Cleaned up re-runnable application design

11.4 Summary

11.5 Extras

11.5.1 Using a SQL database

11.5.2 Persistence with NoSQL databases

Project 3.8: Integrated Data Acquisition Web Service

12.1 Description

12.1.1 The data series resources

12.1.2 Creating data for download

12.2 Overall approach

12.2.1 OpenAPI 3 specification

12.2.2 RESTful API to be queried from a notebook

12.2.3 A POST request starts processing

12.2.4 The GET request for processing status

12.2.5 The GET request for the results

12.2.6 Security considerations

12.3 Deliverables

12.3.1 Acceptance test cases

12.3.2 RESTful API app

12.3.3 Unit test cases

12.4 Summary

12.5 Extras

12.5.1 Add filtering criteria to the POST request

12.5.2 [Split the OpenAPI specification into two parts to use \\$REF for the output schema](#)

12.5.3 [Use Celery instead of concurrent.futures](#)

12.5.4 [Call external processing directly instead of running a subprocess](#)

[Project 4.1: Visual Analysis Techniques](#)

13.1 [Description](#)

13.2 [Overall approach](#)

13.2.1 [General notebook organization](#)

13.2.2 [Python modules for summarizing](#)

13.2.3 [PyPlot graphics](#)

13.2.4 [Iteration and evolution](#)

13.3 [Deliverables](#)

13.3.1 [Unit test](#)

13.3.2 [Acceptance test](#)

13.4 [Summary](#)

13.5 [Extras](#)

13.5.1 [Use Seaborn for plotting](#)

13.5.2 [Adjust color palettes to emphasize key points about the data](#)

Project 4.2: Creating Reports

14.1 Description

14.1.1 Slide decks and presentations

14.1.2 Reports

14.2 Overall approach

14.2.1 Preparing slides

14.2.2 Preparing a report

14.2.3 Creating technical diagrams

14.3 Deliverables

14.4 Summary

14.5 Extras

14.5.1 Written reports with UML diagrams

Project 5.1: Modeling Base Application

15.1 Description

15.2 Approach

15.2.1 Designing a summary app

15.2.2 Describing the distribution

15.2.3 Use cleaned data model

15.2.4 Rethink the data inspection functions

15.2.5 Create new results model

15.3 Deliverables

15.3.1 Acceptance testing

15.3.2 Unit testing

15.3.3 Application secondary feature

15.4 Summary

15.5 Extras

15.5.1 Measures of shape

15.5.2 Creating PDF reports

15.5.3 Serving the HTML report from the data API

Project 5.2: Simple Multivariate Statistics

16.1 Description

16.1.1 Correlation coefficient

16.1.2 Linear regression

16.1.3 Diagrams

16.2 Approach

16.2.1 Statistical computations

16.2.2 Analysis diagrams

16.2.3 Including diagrams in the final document

16.3 Deliverables

16.3.1 Acceptance tests

16.3.2 Unit tests

16.4 Summary

16.5 Extras

16.5.1 Use pandas to compute basic statistics

16.5.2 Use the dask version of pandas

16.5.3 Use numpy for statistics

16.5.4 Use scikit-learn for modeling

16.5.5 Compute the correlation and regression using functional programming

Next Steps

17.1 Overall data wrangling

17.2 The concept of “decision support”

17.3 Concept of metadata and provenance

17.4 Next steps toward machine learning